Task 1 – System Understanding Note

1. What is the main purpose of the application?

Campus Supply Desk is a Python application that manages orders for a campus supply store. Customers can view products, request quotes, place orders, review order history, retrieve specific orders, and cancel purchases. The system checks available inventory before accepting an order and adjusts stock when an order is placed or cancelled.

Prices depend on the selected products, quantities, customer type, and fulfillment method. Students receive a 10% discount when the subtotal reaches $50. Delivery costs $6 unless the amount after discount reaches $100. The application stores inventory and order records in JSON files so information remains available after restarting.

2. What are the major classes, modules, or components, and what does each do?

The repository contains several components:

- **service.py:** Handles business operations, including quotes, order placement, pricing, stock checks, and cancellation.
- **storage.py:** Manages saving and loading application data using JSON files.
- **menu.py:** Supports the interactive terminal interface and displays catalog and order information.
- **cli.py:** Supports individual command-line operations.
- **__main__.py:** Provides the entry point for running python -m supplydesk.
- **catalog.json:** Supplies the initial product catalog, including product IDs, prices, and quantities.
- **test_smoke.py:** Contains basic automated tests for checking application behavior.

These components separate user interaction, order processing, and data storage.

3. Identify one important execution flow and explain how the components work together.

Placing an order is an important execution flow. The customer selects products, enters quantities, and chooses a customer type and fulfillment method through the menu or command line. The request reaches the service layer, which must check the product IDs, quantities, customer type, fulfillment option, and available stock.

After validation, the service calculates line subtotals, any student discount, delivery charges, and the final total. A successful placement receives an order ID beginning with ORD-0001 and the status placed. Stock quantities decrease, and the order and updated inventory are saved. In this expected flow, **menu.py or cli.py handles input, service.py handles ordering rules, and storage.py handles persistence**.

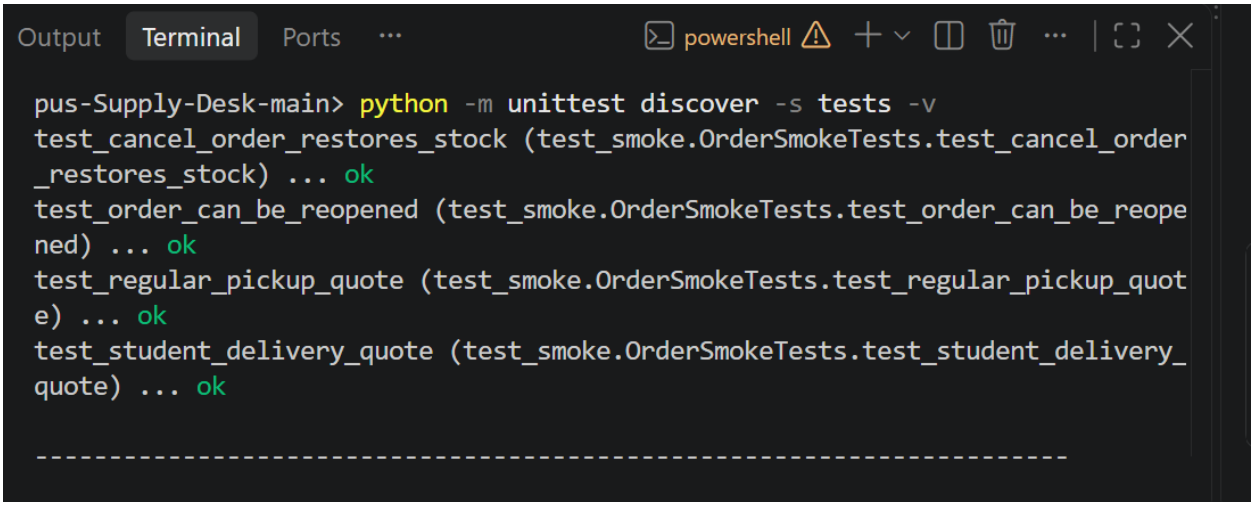
4. Which part of the code appears most difficult to understand or maintain, and why?

The order-processing logic in **service.py** appears to be the area most likely to require careful maintenance because several rules interact. Validation must finish before stock changes, duplicate product lines must be considered together, and quotes must not modify saved information. Pricing also depends on discount and delivery thresholds.

Keeping these responsibilities clearly separated would make changes easier to test and reduce the risk of incorrect totals or inventory updates. However, the exact maintainability problem cannot be confirmed without examining the methods in service.py.

Task 2

1.



Baseline record for Task 2: 4 tests passed, 0 failures, 0 errors.

2.

Checkpoint A – Initial Testing Strategy

My testing strategy will focus on checking the main functions of the Campus Supply Desk application using Python's unittest framework. I plan to test four areas:

1. **Normal Behavior:** Verify that orders containing multiple products are processed correctly and totals are calculated accurately.
2. **Boundary Conditions:** Test the student discount at exactly $50 and free delivery at exactly $100.
3. **Invalid Inputs:** Check that invalid customer types and Boolean quantities are rejected with a ValueError.
4. **Behavior to Preserve:** Verify that order IDs increase sequentially and invalid orders do not change existing orders or inventory.

I will run the original tests before refactoring, add these tests, and record the results. After refactoring, I will run the tests again to check that existing functionality still works correctly.

Task 3

Code Smell	Location	Problem	Evidence	Consequence	Proposed Improvement
Duplicated Code	supplydesk/service.py – OrderService.quote() and OrderService.place_order()	Both methods contain nearly identical code for validating customer information, checking products	Both methods contain nearly identical code for validating customer information, checking products	Repeating the same logic makes maintenance difficult because changes must be made in multiple	Extract the shared validation and pricing calculations into reusable helper methods. Both quote() and

		and quantities, calculating subtotals, and applying discounts and delivery charges.	and quantities, calculating subtotals, and applying discounts and delivery charges.	locations. If one method is updated and the other is missed, quotes and placed orders could produce inconsistent results.	place_order() should use these methods to ensure consistent behavior
Long Method and Multiple Responsibilities	supplydesk/service.py – OrderService.place_order()	The place_order() method handles too many responsibilities within a single method.	It validates customer information, searches for products, checks stock, calculates prices, creates the order dictionary, deducts inventory, increases the order number, and saves the updated state using self.store.save(state).	Combining these operations makes the method harder to understand, debug, and modify. A change to one responsibility, such as inventory management, could unintentionally affect pricing or order creation.	Break the method into smaller functions with clear responsibilities, such as validating requests, preparing order details, and updating inventory. This would improve readability and make individual operations easier to test.
Repeated Product Searching	supplydesk/service.py – OrderService.quote(), OrderService.place_order(), and OrderService.cancel_order()	**:** The application repeatedly searches through the entire product catalog to find products using their IDs.	The methods use loops such as for candidate in state["catalog"] and compare candidate["product_id"] with the requested product ID. In place_order(), products are searched again when stock is deducted, even though they were already located during validation.	Repeated catalog searches make the code unnecessarily complex and increase processing time as the number of products grows. The same lookup logic is also scattered across different operations.	Create a dictionary that maps product IDs to product records. This would allow direct product lookup and reduce repeated loops, making the code simpler and easier to maintain
Missing Test Coverage for Combined Stock Quantities	tests/test_smoke.py and supplydesk/service.py – OrderService.quote() and OrderService.place_order()	The original test suite does not check whether the application correctly handles duplicate product lines whose combined quantities exceed available stock.	The original test_smoke.py contains four tests covering pricing, order persistence, and cancellation. None tests duplicate product lines. In service.py, the condition if quantity > product["stock"] checks each line individually rather than the total	An order containing three canvas bags on one line and two on another could pass validation even though only four bags are available. This could result in negative inventory after placement. Without a test for this situation, the defect could remain	Add automated tests that submit duplicate product lines exceeding available stock. The expected result should be a ValueError, with no changes to inventory, order history, or order numbering. Update validation to check the combined

			requested quantity for a product.	unnoticed or reappear after refactoring.	quantity for each product before processing the order.
Incorrect Student Discount Boundary	supplydesk/service.py – OrderService.quote() and OrderService.place_order()	The student discount condition incorrectly excludes orders with a subtotal of exactly $50.	Both methods use if customer_type == "student" and subtotal > 5000. However, the functional requirements specify that students receive a 10% discount when the subtotal is **at least 5,000 cents**. During baseline testing, test_student_discount_at_threshold failed because the expected discount was 500 cents, but the actual result was 0 cents.	Students placing orders worth exactly $50 are charged more than required. The incorrect condition affects pricing accuracy in both quotes and placed orders	Change the comparison from subtotal > 5000 to subtotal >= 5000 in the shared pricing logic. Keep the existing boundary test to verify that the discount is correctly applied after the change.